



High-Performance E-Commerce Backend engine

مشروع البرمجة التفرعية - السنة الرابعة



عمل الطالبات :
بثينة شكاكي
أنارويده الصويطي
انجلينا قاروط
اسلام الأحمد
مروة الحسن

Contents

Contents

لمحة عن المشروع :	2
المتطلبات الوظيفية :	2
المتطلبات الغير وظيفية :	3
1. حماية البيانات المشتركة من التضارب)	
Concurrent Access & Data Integrity: (	3
2. Resource Management & إدارة الموارد الحاسوبية)	
Capacity Control:(	6
3. Asynchronous Queues: (..	7
المعالجة الغير متزامنة)	
4. Batch معالجة البيانات الضخمة على دفعات)	
Processing(	8
5. Load Distribution(توزيع الأحمال)	9
6. caching Strategy(استراتيجية التخزين المؤقت)	10
7. Distributed Locks(الاقفال الموزعة :)	11
8. Transaction integrity - acid(سلامة المعاملات)	13
9. stress testing	15
10. Bottleneck Analysis(تحليل عنق الزجاجة)	16

لمحة عن المشروع :

- نظام تجارة الكتروني يركز على المفاهيم المتعلقة بالمتطلبات الغير وظيفية يستخدم Django كإطار عمل و PostgreSQL كقاعدة بيانات
- بالإضافة إلى بعض الأدوات الأخرى التي احتجناها لتحقيق المتطلبات
- بالإضافة لأداة JMeter للاختبارات و محاكاة الضغط

المتطلبات الوظيفية :

إبقيناها مبسطة حيث تغطي الحاجة لتطبيق المتطلبات الغير وظيفية

إدارة المنتجات :

عرض منتج : يمكن للمستخدم عرض تفاصيل منتج معين (الاسم و السعر و الكمية)

تحديث المخزون تلقائيا عند الشراء يتم انقاص المخزون بعدد القطع المشتراة

إدارة الحسابات

عرض الرصيد

خصم الرصيد تلقائيا

إدارة الطلبات

إنشاء طلب

عرض سجل الطلبات

معالجة الفواتير

تقارير الدفع

إدارة المستخدمين عن طريق user_id

المتطلبات الغير وظيفية :

و هي أساس المشروع و تم التركيز عليها و اختبارها

1. حماية البيانات المشتركة من التضارب (CONCURRENT ACCESS & DATA INTEGRITY) :

قد تحدث حالة حيث يتم شراء القطعة الأخيرة من منتج مرتين أو حتى بسبب حدوث عملية الشراء في وقت واحد مما قد يؤدي إلى حالة نسميها البيع الزائد overselling مما قد يؤدي إلى دفع سعر منتج غير موجود لأن العملية نجحت رغم عدم توفر المنتج أو بحالة أخرى شراء أكثر من منتج بسعر واحد بسبب التزامن .

هي مشكلة تحدث في الأنظمة متعددة المهام تسمى حالة السباق Race Condition تحدث عندما تحاول مهمتان أو أكثر الوصول إلى مورد مشترك و تكون النتيجة النهائية غير متوقعة

الحل المستخدم : القفل التشاؤمي pessimistic locking -

استخدمنا القفل على مستوى الصف , حيث عند شراء منتج ما محدد يتم قفله حتى انتهاء العمل معه و يمكن فقط لطلب واحد قراءة و تعديل المخزون و بعد أن ينتهي هذا الطلب يفك القفل و يمكن لطلب ثان الشراء في حال توفر كمية كافية

● سبب استخدام القفل التشاؤمي : بسيط يضمن السلامة بدون محاولات متكررة و هو مناسب عندما يكون التضارب متوقع بكثرة كما في المتاجر الإلكترونية

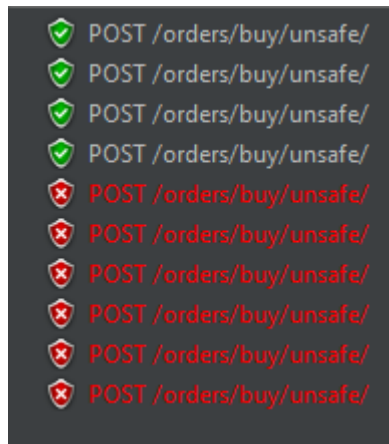
● التطبيق : تم إنشاء تابع يضع قفل على صف المنتج أي طلب آخر يحاول قراءة نفس الصف سياتظر في طابور حتى ينتهي الطلب السابق

بالإضافة إلى إنشاء تابع بدون قفل للاختبار تحت مسمى الشراء الغير الآمن

الاختبار 1 - ظروف الاختبار :

عشر طلبات تحاول شراء نفس المنتج في وقت واحد - الكمية المتاحة من المنتج هي 1 فقط

الشراء الغير آمن :



تمت عملية الشراء أكثر من مرة على منتج غير متوفر و تم خصم سعره من الزبون بعدد عمليات الشراء التي نجحت

الشراء الآمن :



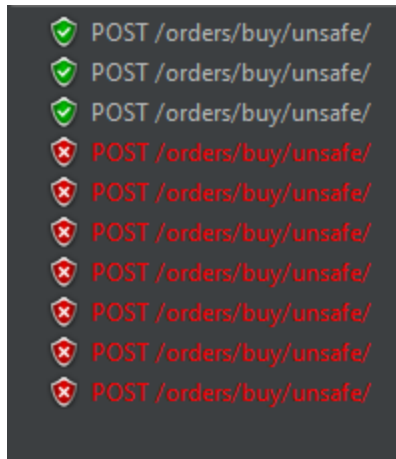
تمت عملية الشراء مرة واحد بطريقة صحيحة و تم رفض باقي الطلبات لعدم توفر المنتج

الاختبار 2 - ظروف الاختبار :

عشر طلبات تحاول شراء نفس المنتج في وقت واحد من مستخدم واحد
كمية كافية من المنتج لكن رصيد يكفي لشراء 1

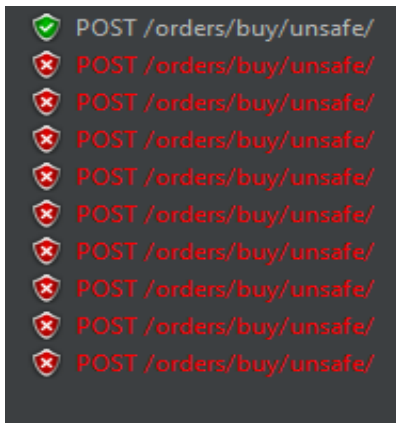
الشراء الغير آمن :

تمت عملية الشراء أكثر من منتج رغم وجود ما يكفي
لشراء واحد فقط



الشراء الآمن :

تمت عملية الشراء مرة واحد بطريقة صحيحة و تم رفض
باقي الطلبات لعدم توفر الرصيد الكافي



2. إدارة الموارد الحاسوبية (RESOURCE MANAGEMENT & CAPACITY CONTROL):

إطار العمل Django هو single_threaded و بالتالي عند وجود الكثير من الطلبات المتزامنة يحدث تكديس و تباطؤ كبير و تعرف أنظمة المتاجر الإلكترونية بالضغط الذي تتعرض له

الحل المستخدم : Fixed Thread Pool

سبب استخدام ال Fixed thread Pool : يمنع انهيار السيرفر لانه مهما وصل عدد الطلبات سيعمل threads ثابتة و تنتظر بقية الطلبات عكس ال cached وهو أيضا مصمم للطلبات الفورية عكس scheduled

التطبيق : استخدمنا خادم waitress مع عدد threads ثابت

الاختبار - ظروف الاختبار :

50 طلب متزامن

single_threaded (Django server) :

نسبة خطأ عالية تحت ضغط عالي
قد تبدو السعة عالية و ذلك لأن الأخطاء تحدث بسرعة و بالتالي لا عبء منها

	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes T
label quest	50	32	0	146	53.58	72.00%	326.8/sec	624.51	22.52	1956.9

fixed thread pool(waitress server) :

نسبة خطأ معدومة و سرعة مناسبة

	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes T
label quest	50	455	65	944	245.52	0.00%	52.6/sec	15.10	12.94	294.0

3. المعالجة الغير متزامنة (ASYNCHRONOUS QUEUES) :

يوجد مهمات ثقيلة قد تبطئ استجابة النظام و في حال قمنا بتنفيذها بشكل متزامن فسيؤدي إلى تأخير الأوامر المستعجلة

و في حال استخدامها لجميع الخيوط قد يتعطل النظام بالكامل

في نظامنا الحالي عند تنفيذ عملية الشراء يجب إرسال فاتورة للعميل و محاكاة إرسالها تستغرق ثانيتين في الكود اذا تم ارسال الفاتورة داخل الطلب مباشرة فإن كل عملية شراء ستجمد الخادم لثانيتين و مع عدد مستخدمين كبير سيكون الأداء سيئاً

الحل المستخدم : Asynchronous Communication

سبب الاستخدام : يرسل الطلب ويكمل العمل ولا يحجب ال thread ولا يعطل النظام التطبيق : عند نجاح عملية الشراء عوضاً عن استدعاء دالة ارسال الفاتورة نضعها في طابور ونعود فوراً لتنفيذ باقي المهام بعدها تنفذ مهمات الطابور

الأدوات المستخدمة :

- مكتبة celery لإدارة المهام الغير متزامنة
- وسيط رسائل يحتفظ بالمهام إلى حين تنفيذها redis

الاختبار - ظروف الاختبار :

عشر طلبات متزامنة

قبل async : نلاحظ بطء النظام والوقت طويل الذي قد يستغرقه

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Sync	10	11152	2079	20218	5789.39	0.00%	29.7/min	0.14	0.12	294.0

مع async :

النظام اسرع بشكل ملحوظ

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Async	10	5640	2116	6319	1333.95	0.00%	1.6/sec	0.45	0.38	294.0

4. معالجة البيانات الضخمة على دفعات (BATCH PROCESSING)

معالجة كميات هائلة من البيانات مثل جميع طلبات اليوم و التي من الممكن أن تكون بكميات هائلة تحميلها دفعة واحدة قد يعطل النظام او يبطئه يستهلك وقت المعالج و يسبب ضغطا

الحل المستخدم : **Batch Processing و الجدولة (Fixed-Size Chunking)**

التجزئة بدل جلب جميع السجلات مرة واحدة يتم جلبها على شكل دفعات batch/chunk فيعالج جزء واحد في الذاكرة ثم ينتقل للتالي كمهام تعمل في الخلفية مما يؤدي إلى استخدام ذاكرة شبه ثابت بغض النظر عن حجم البيانات الأصلي

التطبيق : عند حساب إجمالي المبيعات و عدد الطلبات نقسم الكمية (كل 1000 كدفعة) قبل معالجتها بالاضافة إلى جدولتها عن طريق celery beat و هذا يضمن ان التقرير يولد يوميا دون تدخل يومي

الاختبار :

تم انشاء تابع يمكن استدعاؤه يعمل بدون تقسيم الى دفعات - و اخر يقسم

ظروف الاختبار - يوجد في هذا اليوم **6350** طلب

بدون chunking : ابطا

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
/report/naive/	1	122	122	122	0.00	0.00%	8.2/sec	2.70	1.09	337.0

مع chunking : اسرع بشكل واضح

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
/report/chunked/	1	78	78	78	0.00	0.00%	12.8/sec	4.24	1.73	339.0

5. توزيع الأحمال (LOAD DISTRIBUTION)

عندما يزداد عدد المستخدمين و الطلبات على النظام يصبح وجود سيرفر واحد غير كافي لمعالجة جميع الطلبات في وقت معقول

و كذلك استنزاف الموارد مما قد يؤدي إلى فشل الاتصال

الحل المستخدم : Horizontal Scaling & round_robin

التطبيق :

عوضا عن خادم waitress واحد شغلنا نسختين مستقلتين على المنفذين 8001 و 8002 كلاهما يتصل على نفس قاعدة البيانات ثم تم توزيع الحمل بينهما باستخدام round_robin بالتناوب و هذا يضمن توزيعا متساويا للحمل بالإضافة لتحكم إضافي عن طريق semaphore للحد من الطلبات المتزامنة

سبب استخدام round_robin : المتطلبات المتشابهة من ناحية الوقت لذلك لا حاجة لخوازمية أكثر ذكاء

سبب استخدام Horizontal Scaling : لأنها تعتمد على الكود وتوزيع الاحمال على سيرفرات وهي غير مكلفة ماديا عكس ال vertical

الاختبار:

تم انشاء تابع يمكن استدعاؤه يعمل بدون تقسيم الى دفعات – و اخر يقسم

ظروف الاختبار : 10 طلبات متزامنة

سيرفر واحد :

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	142	68	282	61.58	0.00%	35.3/sec	10.18	8.45	295.0

أكثر من سيرفر :

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	136	84	190	33.75	0.00%	52.6/sec	19.94	12.59	388.0

6. استراتيجية التخزين المؤقت (CACHING STRATEGY)

في أنظمة التجارة الالكترونية, عمليات قراءة تفاصيل المنتجات هي الأكثر تكرار (تصف المستخدمين). كل طلب قراءة يذهب مباشرة الى قاعدة البيانات يؤدي الى

- ضغط متزايد على قاعدة البيانات مع زيادة عدد المستخدمين
- زمن استجابة ابطا لان استعلام قاعدة البيانات (وان كان سريعا) ابطا من القراءة من الذاكرة
- استهلاك موارد غير ضروري عندما تكون البيانات نادرا ماتتغير

الحل المستخدم : Redis – كطبقة تخزين مؤقتة (cach) بين التطبيق وقاعدة البيانات , عند طلب تفاصيل المنتج ,

- اذا كانت البيانات موجودة في Redis , تعاد فورا دون لمس قاعدة البيانات
- اذا لم تكن موجودة, تجلب من PostgreSQL ثم تخزن في Redis لمدة 5 دقائق

طبقتاه هيك:

1. اضفنا اعدادات Redis في settings.py باستخدام قاعدة بيانات منفصلة لعدم التعارض مع Celery
2. انشأنا نقطة نهاية جديدة GET
3. انشأنا نقطة شراء جديدة POST – تقوم بحذف الكاش بعد الشراء لضمن ان المخزون المعروض للمستخدمين محدث دائما
4. ابقينا على النقاط الاصلية POST و GET دون تعديل كمرجع " قبل "

الاختبار :

ظروف الاختبار :

threads : 20

Loop : 10

Cache :

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
Cached	200	36	30	56	92	126	11	151	0.00%	500.0/sec	172.75	94.73
TOTAL	200	36	30	56	92	126	11	151	0.00%	500.0/sec	172.75	94.73

No Cache :

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
No Cache	200	179	178	208	230	285	87	296	0.00%	106.5/sec	32.76	19.45
TOTAL	200	179	178	208	230	285	87	296	0.00%	106.5/sec	32.76	19.45

سرعة أعلى بكثير مع كاش

صحة البيانات : الكاش يحذف بعد الشراء فالمستخدم التالي يرى المخزون الصحيح

7. الاقفال الموزعة : (DISTRIBUTED LOCKS)

إذا كان التنافس العالي (Flash Sales) فقفلة قاعدة البيانات (select_for_update) ينشئ طابور عند قاعدة البيانات نفسها :

- جميع الطلبات (حتى التي تفشل لنفاذ المخزون) تنتظر دورها على القفل
- قاعدة البيانات تصبح عنق الزجاجة
- نحتاج أحيانا لقفل خارج قاعدة البيانات لتخفيف الضغط او للتنسيق بين خدمات متعددة

الحل المستخدم : استخدمنا Redis كقفل موزع (Distributed Lock) لحماية مورد مشترك (كوبونات محدودة) بدون استخدام أي قفل على مستوى قاعدة البيانات. القفل يتم عبر SET NX EX في Redis :

- خيط واحد فقط يستطيع الحصول على القفل في أي لحظة
- باقي الخيوط تنتظر او ترفض بسرعة

طبقاته :

1. انشانا نموذج CouponClaim لتسجيل المطالبين
2. خزنا عدد الكوبونات في Redis (coupon stock = 10)
3. انشانا نقطة نهاية جديدة /POST/orders/coupons/claim/
 - تستخدم redis_client.lock () للحصول على قفل
 - داخل القفل : تقرأ العدد , تتحقق من توفره , تنقصه
 - تحرر القفل القفل
 - تسجل المطالبة في قاعدة البيانات (بدون قفل)
4. ما لمسنا نقاط الشراء الاصلية

الاختبار :

ظروف الاختبار :

عدد الكوبونات 10

20 مستخدم يحاولون الحصول على الكوبونات (threads)

النتيجة :

يحصل المستخدمون العشرة على الكوبونات

و لا يتم تجاوز العدد

مما يدل على سلامة المعاملة تحت الضغط بسبب القفل

SoldOut : القفل الموزع منع البيع الزائد

بالمقارنة مع orders/buy / (قفل قاعدة البيانات) ,

قفل الموزع اسرع ولا يثقل PostgreSQL

8. سلامة المعاملات (TRANSACTION INTEGRITY - ACID)

في الأنظمة المالية (مثل الشراء) , العملية الواحدة تتكون من عدة خطوات : خصم مخزون, خصم رصيد, انشاء, اذا فشلت احدى الخطوات بعد نجاح الأخرى بدون وجود معاملة يحدث عم اتساق في البيانات (Transaction) :

- يخصم المخزون والرصيد لكن لا ينشأ الطلب
- او العكس فيؤدي لخسارة مالية او بيانات فاسدة

الحل المستخدم :

استخدمنا المعاملات في ((Django) transaction.atomic) تضمن مبدأ الكل أو اللاشيء إذا حدث خطأ داخل الكتلة, يتم التراجع عن جميع التغييرات التي تمت داخلها ويعود النظام الى حالته السابقة كما لو لم يحدث شيء

طبقتاه :

1. انشانا نقطة نهاية

بعد : POST/orders/buy/atomic

- كل شيء داخل transaction.atomic()
- نفس التحقق من quantity == 999 يؤدي الى استثناء
- النتيجة : يتم تراجع عن الخصم – المخزون والرصيد ببقايات كما هما – اتساق كامل

الاختبار:

ارسلنا طلب شراء ب quantity = 999

محقق ACID – امانة : المخزون والرصيد لم يتغيرا (تم التراجع)

```
>> curl.exe "http://127.0.0.1:8001/accounts/balance/?user_id=1"
{"id": 1, "name": "laptop", "stock": 10, "price": "500.00"}{"id": 1, "user
name": "user1", "balance": "10000.00"}
(venv) PS C:\Users\night\ecommerce_engine> curl.exe -X POST http://127.0.0
.1:8001/orders/buy/atomic/ -d "user_id=1&product_id=1&quantity=999"
>>
{"error": "Insufficient balance"}
(venv) PS C:\Users\night\ecommerce_engine> curl.exe -X POST http://127.0.0
.1:8001/orders/buy/atomic/ -d "user_id=1&product_id=1&quantity=999"
{"error": "Insufficient balance"}
(venv) PS C:\Users\night\ecommerce_engine> curl.exe http://127.0.0.1:8001/
products/1/
>> curl.exe "http://127.0.0.1:8001/accounts/balance/?user_id=1"
{"id": 1, "name": "laptop", "stock": 10, "price": "500.00"}{"id": 1, "user
name": "user1", "balance": "10000.00"}
(venv) PS C:\Users\night\ecommerce_engine>
```

9.STRESS TESTING

تحت الضغط العالي يجب أن يكون النظام مستقر
و لتحقيق ذلك قمنا باختبار ضغط على النظام

يضم الاختبار أهم العمليات في النظام :
عرض منتج مخزن

عرض منتج غير مخزن

عرض الرصيد

شراء منتج

مطالبة بكميون

سجل الطلبات

ظروف الاختبار :

200 مستخدم (ثريد)

حلقة لا نهائية لمدة دقيقتين

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Cached Product	713	5661	27	64845	12811.05	0.00%	5.6/sec	1.96	0.74	356.0
Balance	707	5708	161	65215	12912.38	0.00%	5.6/sec	1.69	0.78	309.0
Uncached Prod...	693	5261	337	65464	12010.16	0.00%	5.5/sec	1.69	0.68	316.0
Coupon Claim ...	710	6558	386	66661	13465.11	1.55%	5.6/sec	1.62	1.26	295.2
Order History	717	6216	421	65459	13389.33	0.00%	5.7/sec	13193.57	0.78	2380286.1
Optimized Purc...	703	5431	1514	64958	11854.99	0.00%	5.6/sec	1.60	1.39	295.0
TOTAL	4243	5809	27	66661	12768.68	0.26%	33.5/sec	13163.07	5.61	402491.9

ملاحظة النتائج : النظام مستقر بأخطاء قليلة جدا مما يجعله قابلا لتحمل الضغط

يعمل بنسبة 99.74%

10. تحليل عنق الزجاجة (BOTTLENECK ANALYSIS)

تحت الضغط العالي على نفس المنتج, لاحظنا ان قفل الصف (elect_for_update) هو عنق الزجاجة : جميع الطلبات (حتى التي ستفشل) تصطف عند قاعدة البيانات . هذا يسبب

- زمن استجابة مرتفع
- إنتاجية منخفضة
- استهلاك اتصالات قاعدة البيانات في انتظار القفل

الحل المستخدم : قبل الوصول الى قفل قاعدة Redis قمنا بتحسين الأداء عبر فحص مسبق سريع في البيانات :

- نخزن المخزون الحالي في عداد Redis
- قبل الدخول في المعاملة تقرا المخزون من Redis بدون قفل
- اذا كان المخزون غير كاف ترفض الطلب فورا دون لمس قاعدة البيانات
- هذا يقلل عدد الطلبات التي تصل الى القفل ويسرع رفض الطلبات المستحيلة

طبقنا :

1. خزنا مخزون المنتج في id < product_stock (Redis <)
 2. انشانا نقطة نهاية محسنة : /POST/orders/buy/optimized
- تقرا المخزون من redis أولاً
 - اذا كان quantity > تعيد 400 فوراً
 - اذا كان كافياً تكمل العملية العادية ب transaction.atomic()
 - بعد النجاح تحدث قيمة المخزون في Redis

ظروف الاختبار :

5 قطع متبقية

20 طلب لنفس المنتج

Optimaized

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
optimized	20	148	78	195	30.04	75.00%	101.5/sec	28.95	25.28	292.0
TOTAL	20	148	78	195	30.04	75.00%	101.5/sec	28.95	25.28	292.0

bottleneck

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Bottleneck	20	174	83	255	49.54	75.00%	78.4/sec	22.37	18.77	292.0
TOTAL	20	174	83	255	49.54	75.00%	78.4/sec	22.37	18.77	292.0

نلاحظ سرعة أعلى بعد التحسين

شكراً لحضرتكم
